Nikola Knezevic <nikola.knezevic@nikolatech.net>
To: aakashrajarnav@gmail.com

Thu, 14 May 2026 at 10:33

Clean Architecture in ASP.NET Core

Visit Blog

Applications are like children, they're easy to manage while they're small. The problem is, they grow fast.

Then one day you look away for a moment and suddenly your controllers are packed with EF Core queries and business logic.

As the team expands and new developers join, consistency starts to fade. Everyone writes code in slightly different ways just to keep up with delivery pressure. Tests start getting skipped, maintaining coverage becomes harder and introducing changes turns into a nightmare nobody wants to take ownership of.

This is where software architecture becomes important. At its core, software architecture is an agreement between engineers. It defines the shared boundaries, constraints and communication patterns that allow teams to build complex systems in parallel without stepping on each other's work.

One approach to achieving this is Clean Architecture, which focuses on separating core business logic from infrastructure and external implementation details.

Clean Architecture

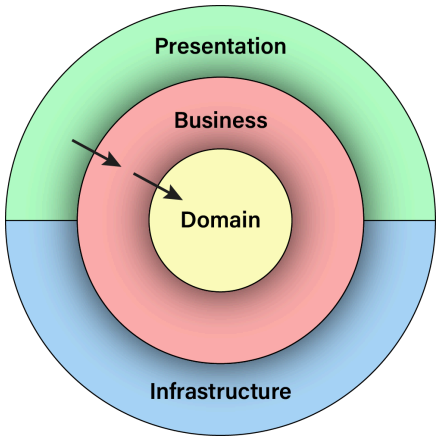
Clean Architecture (popularized by Robert C. Martin) is one of the most structured approaches for building large monolithic applications.

It evolved from ideas such as Hexagonal Architecture, with a strong emphasis on separating business rules from infrastructure concerns. As the domain grows, the goal becomes a clear separation between domain rules and application-level business logic.

Clean Architecture places the domain model at the center of the system.

It aims to solve several common problems that tend to appear in growing applications:

- Poor maintainability
- Business logic leaking into infrastructure
- Database concerns spreading everywhere
- Difficult testing
- Tight coupling to external systems



Clean Architecture addresses these by enforcing strict boundaries between layers. Each layer has a clear responsibility. Business logic becomes testable without external dependencies. Features can evolve independently, and infrastructure changes have minimal impact on the core system.

The dependency rule

The rule that holds everything together is simple: dependencies must point inward.

Inner layers must never depend on outer layers. Instead, outer layers depend on abstractions defined by the inner layers, while concrete implementations and frameworks remain on the

outside. Layers on the same level may depend on each other, but in practice, keeping those dependencies minimal is usually the better choice.

Ideally:

- **Domain** knows nothing
- **Application / Business** depends on domain
- **Infrastructure** depends on application
- **Presentation (WebApi)** depends on everything

Layers in the sample solution

Typical Clean Architecture solution splits projects like this:

- **Domain**: Entities, value objects, domain exceptions, enums and core business rules. The Domain should not have references to other projects.
- **Business**: command/query handlers, services, validators, DTOs and interfaces for infrastructure concerns. Ideally, references only at the project level.
- **Infrastructure**: EF Core, authentication, email providers, file storage, external APIs and other external concerns.
- **WebApi (Presentation)**: minimal endpoints, controllers, DI, swagger, middlewares etc.

```
src
 ├── Domain
 ├── Application
 ├── Infrastructure
 │   ├── Persistence
 │   ├── Authentication
 │   └── ExternalServices
 └── WebApi
```

NOTE: This is not the only valid way to structure a solution, but it is one of the most common and widely adopted approaches. For example, I personally prefer separating Persistence into its own project instead of keeping it inside the Infrastructure layer.

Also, you don't need a separate project for every layer. Projects mainly help enforce boundaries, but the same rules can be followed within a single project if discipline is maintained.

Request flow

Here's how a typical request flow looks like in Clean Architecture:

```
HTTP Request
        ↓
Controller / Endpoint
        ↓
```

```
Handler / Service
        ↓
Domain logic
        ↓
Infrastructure (via abstractions)
```

Handler/Service orchestrates the use case using domain and infrastructure.

Example minimal API:

```csharp
app.MapPost("users/register",
    async (IMediator sender, RegisterRequest request, CancellationToken cancellationToken) =>
{
    var command = request.Adapt<RegisterUserCommand>();

    var response = await sender.SendAsync(command, cancellationToken);

    return Results.Ok(response);
});
```

The request is sent to the handler through a mediator. Before reaching the handler, validation can be executed using pipeline behaviors or validators. If the request is valid, it proceeds to execution.

Example handler:

```csharp
internal sealed class RegisterUserCommandHandler(IApplicationDbContext dbContext, IPasswordHasher
passwordHasher)
    : ICommandHandler<RegisterUserCommand, Guid>
{
    public async Task<Guid> HandleAsync(RegisterUserCommand request, CancellationToken cancellationToken)
    {
        var exists = await dbContext.Users.AnyAsync(x => x.Email == request.Email, cancellationToken);
        if (exists)
        {
            throw new UserWithEmailAlreadyExistsException();
        }

        var hashedPassword = passwordHasher.Hash(request.Password);

        var user = new User(
            Guid.NewGuid(),
            request.Email,
            hashedPassword,
            request.FirstName,
            request.LastName,
            DateTime.UtcNow);

        dbContext.Users.Add(user);

        await dbContext.SaveChangesAsync(cancellationToken);

        return user.Id;
    }
}
```

The handler uses abstractions to interact with data and perform actions. Concrete implementations live in the Infrastructure and Persistence layers. The handler itself only

orchestrates the use case.

Considerations

There is no silver bullet and Clean Architecture is no exception.

It offers many benefits:

- Strong separation of concerns
- Easier testing
- Better scalability
- Reduced coupling
- Flexible infrastructure

However, it also introduces complexity.

Navigation through the solution can feel heavy at first, especially if you are not used to the patterns it introduces. There may be many projects or folders, and abstraction can be abused (not every class needs an interface). The learning curve can also be steep, especially when combined with Domain-Driven Design and other patterns.

It can easily become over-engineered for smaller applications, and not every system needs full Clean Architecture.

On top of that, developers sometimes follow the rules blindly without considering whether they actually add value in a given context.

Conclusion

Clean Architecture is not about adding layers just for the sake of structure.

It is about protecting the core business logic from external complexity.

When applied correctly, it helps teams build systems that are easier to maintain, test, scale and evolve.

The key is balance, apply the principles where they add value, avoid unnecessary complexity and adapt the architecture to the needs of the project.

If you want to check out examples I created, you can find the source code here:

Source Code

I hope you enjoyed it, subscribe and get a notification when a new blog is up!

Visit Newsletter